COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S2
DATE PERFORMED:	August 5-7, 2026	**DATE SUBMITTED:**	August 7, 2026
NAME:	Jaime Luis M. Demain	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

Activity NO. 5.1 **Queues**

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A:

```cpp
#include<iostream>
#include<queue>

void display(std::queue<char>);


int main(){

    //create an object:
    std::queue<char> myQ;



    //use the enqueue operation
    myQ.push('J');
    myQ.push('A');
    myQ.push('I');
    myQ.push('M');
    myQ.push('E');
    std::cout << "current front is: " << myQ.front() << std::endl;
    std::cout << "current back is: " << myQ.back() << std::endl;
    display(myQ);
    //use the dequeue operation
    myQ.pop();
    display(myQ);
    myQ.pop();
    display(myQ);
    myQ.pop();

    //check the queue if it is empty
    std::cout<<"is the queue empty? " << myQ.empty() <<std::endl;
    return 0;
}

void display(std::queue<char> copyQ) {
    //create a copy of the queue
    std::queue<char> temp = copyQ;
    //loop until empty
    while (!temp.empty()){
        //display the front
        std::cout<< " " << temp.front()<<" ";
        //dequeue the front
        temp.pop();
    //add a new line
    std::cout<<std::endl;
    }
}
```

Code

```
PS C:\Users\TIPQC\Documents\CPE 010_Demain_CPE21S2\HOA> cd "c:\Users\TIPQC\Documents\CPE 010_Demain_CPE21S2\HOA\" ; if ($?) { g++ hoa5.cpp -o hoa5 } ; if ($?) { .\hoa5 }
current front is: J
current back is: E
 J
 A
 I
 M
 M
 E
 A
 I
 I
 M
 E
 I
 M
 E
is the queue empty? 0
```

Output

Analysis: This code shows the FIFO operation characteristic of the C++ Standard Template Library queue adaptor std::queue<char>. The queue myQ is created, and the elements 'J', 'A', 'I', 'M', and 'E' are enqueued into it; .front() indicates that 'J' is the oldest element, while .back() indicates that 'E' is the newest one. In order to show the queue elements without changing the original object in main(), display() function accepts a pass-by-value copy and destroys the elements in a while(!temp.empty()) loop by popping each of them from the front to the back. .pop() operations in main() dequeue the elements of myQ from front to back ('J', 'A', 'I'), respectively.

ILO B.1:

```cpp
#include <iostream>
#include "queueLL.h"

int main(){
    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;

    enqueue('J', &front, &back);
    std::cout<<"front: "<< front->data << " " <<"back: "<< back->data<<std::endl;
    enqueue('A', &front, &back);
    std::cout<<"front: "<< front->data << " " <<"back: "<< back->data<<std::endl;
    enqueue('I', &front, &back);
    std::cout<<"front: "<< front->data << " " <<"back: "<< back->data<<std::endl;
    enqueue('M', &front, &back);
    std::cout<<"front: "<< front->data << " " <<"back: "<< back->data<<std::endl;
    enqueue('E', &front, &back);
    std::cout<<"front: "<< front->data << " " <<"back: "<< back->data<<std::endl;

    // Printing all elements in order
    displayAll(front);

    std::cout<<"testing the dequeue operator: \n";
    dequeue(&front, &back);
    std::cout<<"front: "<< front->data << " " <<"back: "<< back->data<<std::endl;
    dequeue(&front, &back);
    std::cout<<"front: "<< front->data << " " <<"back: "<< back->data<<std::endl;
    dequeue(&front, &back);
    dequeue(&front, &back);
    dequeue(&front, &back);
    dequeue(&front, &back);

    // Checking if empty
    if (isEmpty(front)) {
    std::cout << "The queue is currently empty.\n";
    }
    return 0;

}
```

Main.cpp

```cpp
#ifndef QUEUELL_H
#define QUEUELL_H

//create a class node
template<typename T>
class qNode{
    public:
    T data;
    qNode* next;
};
//new node creator
template<typename T>
qNode<T>* new_node(T newData){
    //allocate a space for the new data
    qNode<T>* newNode = new qNode<T>;

    //store the newData to the newnode
    newNode -> data = newData;

    //point the newNode next to null
    newNode -> next = nullptr;

    return newNode;


}
//enqueue
template <typename T>
void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr ){
    //create new node
    qNode<T>* newNode = new_node(newData);

    //check if the queue is empty
    if(*frontPtr ==nullptr && (*backPtr)==nullptr){
        (*frontPtr) = newNode;
        (*backPtr) = newNode;
        return;
    }

    //if not then insert at the back
    (*backPtr) -> next = newNode;
    (*backPtr) = newNode;
}

//dequeue
template <typename T>
void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){
    //check if the queue is empty
    if (*frontPtr == nullptr && *backPtr == nullptr){
        std::cout << "the queue is empty \n";
        return;
    }
    //create a temporary variable to store the node to be deleted
    qNode<T>* deleteTemp = nullptr;

    //assign the current front to the deleteTemp
    deleteTemp = (*frontPtr);

    //check if the queue has only 1 item
```

```cpp
    if((*frontPtr) -> next == nullptr && (*backPtr) -> next == nullptr){
        //means only 1 item is inside the queue
        (*frontPtr) = nullptr;
        (*backPtr) = nullptr;
        delete deleteTemp;
        return;
    }
    (*frontPtr) =  (*frontPtr)-> next;
    deleteTemp->next = nullptr;
    //delete node
    delete deleteTemp;
}
//display all
template <typename T>
void displayAll(qNode<T>* frontPtr) {
    if (isEmpty(frontPtr)) {
        std::cout << "Queue is empty." << std::endl;
        return;
    }

    qNode<T>* temp = frontPtr;
    std::cout << "Queue elements: ";
    while (temp != nullptr) {
        std::cout << temp->data << " ";
        temp = temp->next;
    }
    std::cout << std::endl;
}
//isEmpty
template <typename T>
bool isEmpty(qNode<T>* frontPtr) {
    return frontPtr == nullptr;
}

#endif
```

Header file

```
PS C:\Users\TIPQC\Documents\CPE 010_Demain_CPE21S2\HOA> cd "c:\Users\TIPQC\Documents\CPE 010_Demain_CPE21S2\HOA\" ; if ($?) { g++ hoa5b.cpp -o hoa5b } ; if ($?) { .\hoa5b }
front: J back: J
front: J back: A
front: J back: I
front: J back: M
front: J back: E
front: J back: E
Queue elements: J A I M E
testing the dequeue operator:
front: A back: E
front: I back: E
the queue is empty
front: A back: E
front: I back: E
the queue is empty
front: I back: E
the queue is empty
The queue is currently empty.
PS C:\Users\TIPQC\Documents\CPE 010_Demain_CPE21S2\HOA>
```

Output

Analysis: The implementation is done using the queue template based program and uses double pointers (qNode<T>**) in order to manipulate the memory directly. The implementation within queueLL.h takes care of allocating memory dynamically, queuing of items following FIFO fashion at backPtr and deleting of nodes using delete command at frontPtr in order to avoid any memory leakage. In main.cpp, the test driver queues the items J,A,I,M,E, displays the queue, removes one item at a time and finally confirms the empty queue using isEmpty().

ILO B.2:

```cpp
#include <iostream>
#include "queueARR.h"

int main(){

    queueArr<int> q(5);

    // enqueue 10, 20, 30 ,40, 50:
    q.Enqueue(1);

    std::cout<<"Front: " << q.Front() <<std::endl;
    std::cout<<"Back: " << q.Back() <<std::endl;
    std::cout<<"Size: " << q.Size() <<std::endl;
    q.Enqueue(2);
    std::cout<<"Front: " << q.Front() <<std::endl;
    std::cout<<"Back: " << q.Back() <<std::endl;
    std::cout<<"Size: " << q.Size() <<std::endl;
    q.Enqueue(3);
    q.Enqueue(4);
    std::cout<<"Front: " << q.Front() <<std::endl;
    std::cout<<"Back: " << q.Back() <<std::endl;
    std::cout<<"Size: " << q.Size() <<std::endl;
    q.Enqueue(5);
    std::cout<<"Front: " << q.Front() <<std::endl;
    std::cout<<"Back: " << q.Back() <<std::endl;
    std::cout<<"Size: " << q.Size() <<std::endl;
    q.Enqueue(6);

    std::cout << "\nremoved: " << q.Dequeue() <<std::endl;
    std::cout << "removed: " << q.Dequeue() <<std::endl;

    //testing the clear
    q.Clear();

    std::cout << "is the queue empty? " << (q.Empty() ? "yes": "no") <<std::endl;

    return 0;
}
```

Main.cpp

```cpp
#ifndef QUEUEARR_H
#define QUEUEARR_H
#include <iostream>

template <typename T>
class queueArr {
private:
    // Pointer to dynamically allocated array
    T* q_array;
    // Maximum number of elements a queue can hold
    size_t q_capacity;
    // Current number of elements in the queue
    size_t q_size;
    // Index of the front element
    int q_front;
    // Index of the back element
    int q_back;

public:
    // Constructor
    queueArr(size_t capacity = 10);
    // Copy Constructor
    queueArr(const queueArr& other);
    // Copy Assignment Operator
    queueArr& operator=(const queueArr& other);
    // Destructor
    ~queueArr();

    // Queue Operations
    bool Empty();
    bool Full();
    size_t Size();
    void Clear();
    T Front();
    T Back();
    void Enqueue(T value);
    T Dequeue();
};

// Constructor
template <typename T>
queueArr<T>::queueArr(size_t capacity) {
    q_capacity = capacity;
    q_array = new T[q_capacity];
    q_size = 0;
    q_front = 0;
    q_back = -1;
}

// Copy Constructor
template <typename T>
queueArr<T>::queueArr(const queueArr& other) {
    q_capacity = other.q_capacity;
    q_size = other.q_size;
    q_front = other.q_front;
    q_back = other.q_back;
    q_array = new T[q_capacity];
    for (size_t i = 0; i < q_capacity; i++) {
        q_array[i] = other.q_array[i];
    }
}

// Copy Assignment Operator
template <typename T>
queueArr<T>& queueArr<T>::operator=(const queueArr& other) {
    if (this != &other) {
        delete[] q_array;
        q_capacity = other.q_capacity;
        q_size = other.q_size;
        q_front = other.q_front;
        q_back = other.q_back;
```

```
72          q_array = new T[q_capacity];
73          for (size_t i = 0; i < q_capacity; i++) {
74              q_array[i] = other.q_array[i];
75          }
76      }
77      return *this;
78  }
79
80  // Destructor
81  template <typename T>
82  queueArr<T>::~queueArr() {
83      delete[] q_array;
84  }
85
86  // Empty()
87  template <typename T>
88  bool queueArr<T>::Empty() {
89      return q_size == 0;
90  }
91
92  // Full()
93  template <typename T>
94  bool queueArr<T>::Full() {
95      return q_size == q_capacity;
96  }
97
98  // Size()
99  template <typename T>
100 size_t queueArr<T>::Size() {
101     return q_size;
102 }
103
104 // Front()
105 template <typename T>
106 T queueArr<T>::Front() {
107     if (Empty()) {
108         std::cout << "Queue is Empty\n";
109         return T();
110     }
111     return q_array[q_front];
112 }
113
114 // Back()
115 template <typename T>
116 T queueArr<T>::Back() {
117     if (Empty()) {
118         std::cout << "Queue is empty\n";
119         return T();
120     }
121     return q_array[q_back];
122 }
123
124 // Clear()
125 template <typename T>
126 void queueArr<T>::Clear() {
127     q_size = 0;
128     q_front = 0;
129     q_back = -1;
130 }
131
132 // Enqueue()
133 template <typename T>
134 void queueArr<T>::Enqueue(T value) {
135     // Check if the queue is FULL
136     if (Full()) {
137         std::cout << "Queue Overflow! Cannot enqueue " << value << "\n";
138         return;
139     }
```

```
    // Move q_back circularly
    q_back = (q_back + 1) % q_capacity;
    // Store the value at the back
    q_array[q_back] = value;
    // Increment the q_size
    q_size++;
}

// Dequeue()
template <typename T>
T queueArr<T>::Dequeue() {
    // Check if empty
    if (Empty()) {
        std::cout << "Queue Underflow! Cannot dequeue\n";
        return T();
    }
    // Create a temporary variable to store the current front
    T tempVar = q_array[q_front];

    // Move q_front circularly
    q_front = (q_front + 1) % q_capacity;

    // Decrement the q_size
    q_size--;

    // Reset the indexes if the queue becomes empty
    if (Empty()) Clear();

    // Return the temporary variable
    return tempVar;
}

#endif
```

Header file

```
PS C:\Users\jaime\OneDrive\Documents\HOA\hoa5> cd "c:\Users\jaime\OneDrive\Documents\HOA\hoa5\" ; if ($?) { g++ hoa52.cpp -o hoa52 } ; if ($?) { .\hoa52 }
Front: 1
Back: 1
Size: 1
Front: 1
Back: 2
Size: 2
Front: 1
Back: 4
Size: 4
Front: 1
Back: 5
Size: 5
Queue Overflow! Cannot enqueue 6

removed: 1
removed: 2
is the queue empty? yes
```

Output

Analysis: The main.cpp file serves as a test driver to test out the functional aspects of the First In First Out mechanics, boundaries and state management in the user-defined circular array queue. First it initializes the queueArr type with a size of five elements, enqueues integers into it and prints out the Front(), Back(), and Size() to demonstrate that the front element is always the oldest element in the queue despite addition of new elements in the rear of the queue. It goes on to purposely trigger the overflow guard clause when it tries to enqueue the sixth element (q.Enqueue(6)) showing that it

properly prevents any extra elements from being added without triggering a run-time error. This is followed by several calls of Dequeue() to demonstrate extraction in FIFO order, and q.Clear() to reset the indices and the size back to zero.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

```
#include <iostream>
#include <string>

// Step 1: Job Class
class Job {
public:
    int jobID;
    std::string userName;
    int numberOfPages;

    Job(int id, std::string user, int pages)
        : jobID(id), userName(user), numberOfPages(pages) {}

    void displayJob() const {
        std::cout << "Job #" << jobID << " | User: " << userName
                  << " | Pages: " << numberOfPages;
    }
};

// Node structure for custom queue
struct JobNode {
    Job job;
    JobNode* next;
    JobNode(Job j) : job(j), next(nullptr) {}
};

// Step 2 & 3: Printer Class with Custom Linked List Queue
class Printer {
private:
    JobNode* frontPtr;
```

```
    JobNode* backPtr;
    int pendingCount;

public:
    Printer() : frontPtr(nullptr), backPtr(nullptr), pendingCount(0) {}

    ~Printer() {
        while (frontPtr != nullptr) {
            JobNode* temp = frontPtr;
            frontPtr = frontPtr->next;
            delete temp;
        }
    }

    // Add a new print job to the queue
    void addJob(int id, std::string user, int pages) {
        Job newJob(id, user, pages);
        JobNode* newNode = new JobNode(newJob);

        if (frontPtr == nullptr) {
            frontPtr = newNode;
            backPtr = newNode;
        }
        else {
            backPtr->next = newNode;
            backPtr = newNode;
        }
        pendingCount++;

        std::cout << "[ADDED] ";
        newJob.displayJob();
        std::cout << "\n";
    }

    // Process all jobs in First-Come, First-Served order
    void processJobs() {
        std::cout << "\n--- STARTING PRINTER JOB PROCESSING ---\n";
        if (frontPtr == nullptr) {
```

```
            std::cout << "No pending print jobs in the queue.\n";
            return;
        }

        while (frontPtr != nullptr) {
            JobNode* temp = frontPtr;
            std::cout << "[PROCESSING] ";
            temp->job.displayJob();
            std::cout << " -> Completed!\n";

            frontPtr = frontPtr->next;
            delete temp;
            pendingCount--;
        }
        backPtr = nullptr;
        std::cout << "All print jobs completed successfully.\n";
    }

    void showPendingJobs() const {
        std::cout << "\n--- CURRENT PENDING JOBS (" << pendingCount << ")
---\n";
        if (frontPtr == nullptr) {
            std::cout << "Printer queue is empty.\n";
            return;
        }
        JobNode* curr = frontPtr;
        while (curr != nullptr) {
            std::cout << " -> ";
            curr->job.displayJob();
            std::cout << "\n";
            curr = curr->next;
        }
    }
};

// Step 4: Scenario Simulation
int main() {
    Printer officePrinter;
```

```
    std::cout << "=== SHARED PRINTER SIMULATION ===\n\n";

    // Users submitting jobs
    officePrinter.addJob(101, "Engr. Jaime", 6);
    officePrinter.addJob(102, "Engr. Jammy", 7);
    officePrinter.addJob(103, "Engr. Jaimito", 19);
    officePrinter.addJob(104, "Engr. Tempest", 5);

    // View queue before processing
    officePrinter.showPendingJobs();

    // Process all jobs
    officePrinter.processJobs();

    // Verify empty queue
    officePrinter.showPendingJobs();

    return 0;
}
```

Code

Why did I choose **a linked list?**

Answer: For the purpose of implementing the shared printer simulation, I decided to use the singly linked list data structure for internal representation, as it supports dynamic memory allocation along with the best $O(1)$ performance of queue operations. In contrast to the circular array with a fixed capacity that may cause an overflow in case of high print job volume or wasting allocated memory when not used, the linked list allocates each JobNode object in the heap as required and deallocates right after processing. Moreover, deletion of processed elements from the beginning of the queue is done in $O(1)$ by just setting the pointer to the next element and calling the delete method.

```
PS C:\Users\jaime\OneDrive\Documents\HOA\hoa5> cd "c:\Users\jaime\OneDrive\Documents\HOA\hoa5\" ; if ($?) { g++ mainsupp.cpp -o mainsupp } ; if ($?) { .\mainsupp }
=== SHARED PRINTER SIMULATION ===

[ADDED] Job #101 | User: Engr. Jaime | Pages: 6
[ADDED] Job #102 | User: Engr. Jammy | Pages: 7
[ADDED] Job #103 | User: Engr. Jaimito | Pages: 19
[ADDED] Job #104 | User: Engr. Tempest | Pages: 5

--- CURRENT PENDING JOBS (4) ---
 -> Job #101 | User: Engr. Jaime | Pages: 6
 -> Job #102 | User: Engr. Jammy | Pages: 7
 -> Job #103 | User: Engr. Jaimito | Pages: 19
 -> Job #104 | User: Engr. Tempest | Pages: 5

--- STARTING PRINTER JOB PROCESSING ---
[PROCESSING] Job #101 | User: Engr. Jaime | Pages: 6 -> Completed!
[PROCESSING] Job #102 | User: Engr. Jammy | Pages: 7 -> Completed!
[PROCESSING] Job #103 | User: Engr. Jaimito | Pages: 19 -> Completed!
[PROCESSING] Job #104 | User: Engr. Tempest | Pages: 5 -> Completed!
All print jobs completed successfully.

--- CURRENT PENDING JOBS (0) ---
Printer queue is empty.
```

Output

Analysis: The output given above clearly shows how the First-In, First-Out (FIFO) property of the Queue ADT is followed in the case of the First-Come, First-Served (FCFS) shared printer system. In the enqueue operation, the print job requests of #101 to #104 are dynamically added at the end of the queue using backPtr, thereby creating a linked list of jobs in which the order of their arrival is maintained. The pending status check then iterates over the linked list starting from frontPtr down to nullptr without changing any data, proving that all four jobs are stored in the memory in a sequential manner. The process of printing begins when the printer starts dequeuing the jobs from the head of the queue (frontPtr) and finishes Job #101 first. Here, frontPtr moves to the next node (frontPtr = frontPtr->next) and dynamic memory of the heap is deleted using the delete operator. The procedure continues in a similar fashion for Job #102, #103, and #104 until the entire list becomes empty (both frontPtr and backPtr are nullptr).

Conclusions: From this activity, I have gained knowledge regarding Queue Abstract Data Type implementation using C++ Standard Template Library, circular array, and linked list based on the First In First Out (FIFO) algorithm. Through the process of the analysis, it is clear that there were differences between standard library data containers and manual pointer manipulations, and it is important to assign frontPtr and backPtr precisely and utilize delete command to avoid memory leak problem when dequeuing from the queue. Besides that, the supplementary printer problem has proved that using linked list queue can model real life FCFS task scheduling without facing any problem of array capacity and extra shifting operations. Overall, I have done excellent in creating the required template structure and passing pointer-to-pointer parameter.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, Quillbot, etc.

COMPUTER ENGINEERING

QUEZON CITY

Purpose of Use: **For code assistance in supplementary activity to understand it and improving grammar in forming an observations, analysis, and conclusion.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used AI to help me understand the supplementary activity. It's because I need to have more clarity on how the supplementary can be formed so that I can make the supplementary activity neat and working. Also, I used AI to improve my grammar and tone in forming my observations of the ILOs , the analysis on its code part, and in conclusion.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.